



JAVA PROGRAMMING

(340)

NATIONAL 2022

APPLICATION KNOWLEDGE:

Tollway Customer Database National _____ (420 points)

TOTAL POINTS _____ (*420 points*)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should not appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may not be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter.

The graders will not compile or alter your source code to correct for this.
Submissions that do not contain source code will not be graded.

Assumptions to make when taking this assessment:

- The text file imported into the driver program will be error free and all of the data values will be in the correct order. NOTE: the product type symbol can be either upper or lowercase.
- The input file will contain only ASCII characters.
- A test input file will be available and named, "portfolioText.txt."
- The Stocks.class file needs to be in your project folder and will be used to help with the solution. This class will be used to construct Stock objects in the Portfolio.
- The InvestmentProducts.java file needs to be in the project and will be used to help create a child class called ETF.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).
- If you create a class, then you must use Javadoc comments.

Note to Graders: The contestants have been given a file named "Names.txt" and starter file called "TollwayCustomerDataBaseNational.java". You must first delete the "Names.txt" file from the contestant's flash drive and run the program for Test case #1. This should result in a message that the file was not found. Then replace the file on their flash drive with the grader's file with the same name and run Test case #2. You'll run their program with a grader's file named "Names.txt" which will contain different data than the student file. This will produce different results than the student's file. You must copy the grader's "Names.txt" file into the folder where their program runs. Listed below are the test cases and their results.

Test case #1 verifies that the program checks for the correct input file. The output wording does *not* need to be exact.

Text case #2 processes the correct data file name and produces output. The output wording *must* be exact; however, the object memory address will be proprietary to the machine running the code. The list is broken into two parts: top half are the original imported records from Names.txt, and the bottom half is the cloned records.

Text case #3 the grades will check the written Clones.txt file created by the program which will contain only the cloned objects. The top of the text file should have the current date printed at the top of the file for when the “Clones.txt” was created.

Test Case #1

The file doesn't exist.

Test Case #2.a (top half of imported records from Names.txt: note this is just a sample there are 12 total)

Customer@12edcd21

Name: Jose Montana

Car Information: Ford F150 ABC-123

Payment: \$65.00 WHJFHJFO483897

Customer@7eda2dbb

Name: Isaac Newton

Car Information: Chevy Silverado DFR-345

Payment: \$89.01 WH24223442897

Test Case #2.b (bottom half of imported records are cloned from the originals: note this is just a sample there are 12 total, and additional formatting is used to quickly ID clone records.)

*****CLONES*****

Customer@1a6c5a9e

DUPLICATES

CLONED Name: Jose Montana

CLONED Car Information: Ford F150 ABC-123

CLONED Payment: \$65.00 WHJFHJFO483897

Customer@37bba400

DUPLICATES

CLONED Name: Isaac Newton

CLONED Car Information: Chevy Silverado DFR-345

CLONED Payment: \$89.01 WH24223442897

Test Case #3 (the exported Clones.txt file will be an exact copy of the Test Case #2.b; take note that there is a current date printed everytime the program runs, this is printed at the top per the student requirements.)

Mon Nov 23 14:44:05 CST 2020

DUPLICATES

CLONED Name: Jose Montana

CLONED Car Information: Ford F150 ABC-123

CLONED Payment: \$65.00 WHJFHJFO483897

DUPLICATES

CLONED Name: Isaac Newton

CLONED Car Information: Chevy Silverado DFR-345

CLONED Payment: \$89.01 WH24223442897

Your application will be graded on the following criteria:		
Solution and Project		
The project is present on the flash drive		10 points
The projects main class is named TollwayCustomerDataBaseNational		10 points
The projects helper class is named Customer		10 points
The projects helper class is named Name		10 points
The projects helper class is named CarInfo		10 points
The projects helper class is named Payment		10 points
The interface is called InformationRetrieval		10 points
Program Execution		
The program runs from the USB flash drive	_____	10 points
If the program does <i>not</i> execute, then the remaining items in this section receive a score of zero.		
The program displays in the console if the “Names.txt” file being read is not found: “The file doesn't exist.”	_____	10 points
The program displays the entire set of records imported from “Names.txt” in the proper formatting per the instructions (12 total records)	_____	10 points
The program clones and displays the cloned records in the proper formatting per the instructions (12 total records)	_____	30 points
Program writes only the cloned records to a text file in the proper format; file must be called “Clones.txt”	_____	40 points
The “Clones.txt” contains the current date and time at the top of the document for every time it is written	_____	20 points
Source Code Review		
The source code is properly commented		
A comment containing the contestant number is present	_____	10 points
Methods and code sections are commented	_____	20 points
Helper Classes each implement InformationRetrieval interface (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Helper Classes each implement Cloneable interface (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Helper Classes each use InformationRetrieval abstract method confirmed() ; properly (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Helper Classes each use InformationRetrieval abstract method getInfo() ; properly (5 points each): Customer, Name, CarInfo, Payment	_____	20 points

Helper Classes each use Cloneable abstract method <i>clone()</i> ; properly (5 points each): Customer, Name, CarInfo, Payment	_____	20 points
Customer objects read from “Names.txt” are stored in a List	_____	10 points
Cloned customer objects are stored in a separate List	_____	10 points
Code uses try... catch for FileNotFoundException exception handling in TollwayCustomerDataBaseState class when reading the file; commenting must be used to identify.	_____	20 points
Code uses try... catch for IOException exception handling in TollwayCustomerDataBaseState class when inputting from the file; commenting must be used to identify.	_____	20 points
Code uses try... catch for IOException exception handling in TollwayCustomerDataBaseState class when writing to a file; commenting must be used to identify.	_____	20 points
<i>Object clone()</i> : Code uses try... catch for exception handling in all helper class implementing Cloneable interface (catches <i>CloneNotSupportedException</i>) 5 points each	_____	20 points

**Total Points = _____ / 420
points**

Suggested Solution

```
1 import java.util.*;
2 import java.text.NumberFormat;
3 import java.io.*;
4
5 |
6 public class TollwayCustomerDataBaseNational
7 {
8
9
10 public static void main (String args [])
11 {
12
13
14     BufferedReader in = getReader("Names.txt");
15     List<Customer> customers = new ArrayList<>(); //Contains the customer obj
16     List<Customer> clonedCustomers = new ArrayList<>();
17     Customer cust = readCustomer(in);
18     customers.add(cust);
19     while (cust != null)
20     {
21         cust = readCustomer(in);
22         if(cust != null)
23             customers.add(cust);
24     }
25     for (Customer c : customers){
26         System.out.println(c);
27         System.out.println(c.getInfo());
28         clonedCustomers.add((Customer)c.clone());
29     }
30     System.out.println("*****CLONES*****");
31     for (Customer c : clonedCustomers){
32         System.out.println(c);
33         System.out.println(c.getInfo());
34     }
35
36     //This block will export the data to the text file.
37     PrintWriter out = openWriter("Clones.txt");
38     exportDate(out);
39     for (Customer c : clonedCustomers)
40         exportClones(c, out);
41     out.close();
42 }
43
44 //These two methods write to the file. exportDate prints the current date at t
45 private static void exportDate(PrintWriter out)
46 {
47     java.util.Date date = new java.util.Date();
48     out.println(date);
49 }
50
51 //exportClones prints each cloned Customer record to the text file
52 private static void exportClones(Customer c,
53     PrintWriter out)
54 {
55     String line = c.getInfo();
56     out.println(line);
57 }
58
```



```
58
59 //Creates the proper file for the text to be written to
60 private static PrintWriter openWriter(String name)
61 {
62     try
63     {
64         File file = new File(name);
65         PrintWriter out =
66             new PrintWriter(
67                 new BufferedWriter(
68                     new FileWriter(file), true));
69         return out;
70     }
71     catch (IOException e)
72     {
73         System.out.println("I/O Error");
74         System.exit(0);
75     }
76     return null;
77 }
78
79
80 //Reads the information from the file and breaks it into each of the objects that go into the Customer object
81 private static Customer readCustomer(BufferedReader in)
82 {
83     //Name Class plus Fields
84     Name name; String firstN; String lastN;
85     //CarInfo Class plus Fields
86     CarInfo carinfo; String make; String model; String plates;
87     //Payment Class plus Fields
88     Payment payment; double deposit; String creditcard;
89     String line = "";
90     String[] data;
91
92     //catches errors in the read
93     try
94     {
95         line = in.readLine();
96     }
97     catch (IOException e)
98     {
99         System.out.println("I/O Error");
100         System.exit(0);
101     }
102     //ends the read once it gets to the end
103     if (line == null)
104         return null;
105     else
106     {
107         data = line.split(","); //"," is the delimiter to separate the fields for the records
108         firstN = data[0]; //each field goes in escalating arithmetic sequence 0 to 6
109         lastN = data[1];
110         make = data[2];
111         model = data[3];
112         plates = data[4];
113         deposit = Double.parseDouble(data[5]);
114         creditcard = data[6];
115         //construct the objects to be placed in the Customer object
116         name = new Name(firstN, lastN);
117         carinfo = new CarInfo(make, model, plates);
118         payment = new Payment(deposit, creditcard);
119         return new Customer(name, carinfo, payment);
120     }
121 }
122
```

```

122
123
124
125 //imports the file properly for BufferedReader
126 private static BufferedReader getReader(String name)
127 {
128     BufferedReader in = null;
129     try
130     {
131         File file = new File(name);
132         in = new BufferedReader(
133             new FileReader(file) );
134     }
135     catch (FileNotFoundException e)
136     {
137         System.out.println(
138             "The file doesn't exist.");
139         System.exit(0);
140     }
141     return in;
142 }
143 }
144
149
150 //Customer Objects Information////////////////////////////////////
151 class Customer implements InformationRetrieval, Cloneable
152 {
153     public Name name;
154     public CarInfo carinfo;
155     public Payment pay;
156     private String cloned = ""; //this is just a flag to help quickly identify if the record was cloned
157
158     public Customer(Name n, CarInfo c, Payment p)
159     {
160         this.name = n;
161         this.carinfo = c;
162         this.pay = p;
163     }
164
165     //Required method from the interface
166     public String getInfo()
167     {
168
169         return cloned + this.name.getInfo() + "\n" + this.carinfo.getInfo() + "\n" + this.pay.getInfo() + "\n" ;
170     }
171
172     //Required method from the interface
173     public Object clone()
174     {
175         Customer customer;
176         try
177         {
178             cloned = confirmed();
179             customer = (Customer) super.clone();
180             customer.name = (Name)name.clone();
181             customer.carinfo = (CarInfo)carinfo.clone();
182             customer.pay = (Payment)pay.clone();
183         }
184         catch (CloneNotSupportedException e)
185         {
186             return null;
187         }
188         return customer;
189     }
190     // This method only supplies the cloned variable with a string literal. It is only called in the clone() method
191     public String confirmed()
192     {
193         return "DUPLICATES \n";
194     };
195 }
196

```

```
196
197 //////////////Name Object Information////////////////////////
198 class Name implements InformationRetrieval, Cloneable
199 {
200     private String last_Name;
201     private String first_Name;
202     private String cloned = "";
203
204     public Name(String fn, String ln)
205     {
206         this.last_Name = ln;
207         this.first_Name = fn;
208     }
209
210     public String getInfo()
211     {
212         return cloned + " Name: " + this.first_Name + " " + this.last_Name;
213     }
214     public Object clone()
215     {
216         try
217         {
218             cloned = confirmed();
219             return super.clone();
220         }
221         catch (CloneNotSupportedException e)
222         {
223             return null;
224         }
225     }
226
227     public String confirmed()
228     {
229         return "CLONED";
230     };
231
232 }
233
234
```

[illegible]